

Depth First Search (DFS)

Aim

To implement the **Depth-First Search (DFS)** algorithm in Python to traverse all the vertices of a graph using recursion.

Algorithm

1. Create a graph using an adjacency list.
2. Choose a starting vertex.
3. Mark the starting vertex as visited.
4. Visit the current vertex and print it.
5. Recursively visit each unvisited adjacent vertex.
6. Continue until all reachable vertices are visited.

Python Code:

```
graph = {}

# Number of vertices
n = int(input("Enter the number of vertices: "))

# Input graph
for i in range(n):
    vertex = input(f"Enter vertex {i+1}: ")
    neighbours = input(f"Enter neighbours of {vertex} (space-separated): ").split()
    graph[vertex] = neighbours

start = input("Enter the starting vertex: ")

visited = set()
```

```

def dfs(vertex):
    if vertex not in visited:
        print(vertex, end=" ")
        visited.add(vertex)

    for neighbour in graph[vertex]:
        dfs(neighbour)

print("\nDFS Traversal:")
dfs(start)

```

Output:

```

Enter the number of vertices: 5
Enter vertex 1: a
Enter neighbours of a (space-separated): b c
Enter vertex 2: b
Enter neighbours of b (space-separated): d
Enter vertex 3: d
Enter neighbours of d (space-separated): e
Enter vertex 4: c
Enter neighbours of c (space-separated):
Enter vertex 5: e
Enter neighbours of e (space-separated):
Enter the starting vertex: a

DFS Traversal:
a b d e c

```

Result

The **Depth-First Search (DFS)** algorithm was successfully implemented in Python. The program traversed the graph by visiting each vertex as deeply as possible before backtracking, and all reachable vertices were visited successfully.